



中南大學

CENTRAL SOUTH UNIVERSITY

高级程序设计 课程设计

Course Design of Advanced Programming

题 目： 证券投资决策支持系统

学生姓名： 张瑞杨

指导教师： 唐朝晖

学 院： 自动化学院

专业班级： 自动化与电气类 2518 班

完成时间： 2026 年 1 月 21 日

目录

1. 课程设计基本目的及要求
2. 软件整体规划及设计
 - (1) 流程图
 - (2) 类的设计
3. 程序详细设计
 - 一. 登录界面
 - 二. 菜单界面
 - (1) 账户管理
 - (2) 市场数据
 - (3) 股票数据
 - (4) 投资分析
 - (5) 投资预测
 - (6) 退出系统
 - (7) 其他辅助函数
4. 程序测试
5. 软件现存问题与不足
6. 设计体会与未来的展望
7. 参考文献

1. 本次课程设计的基本目的和要求

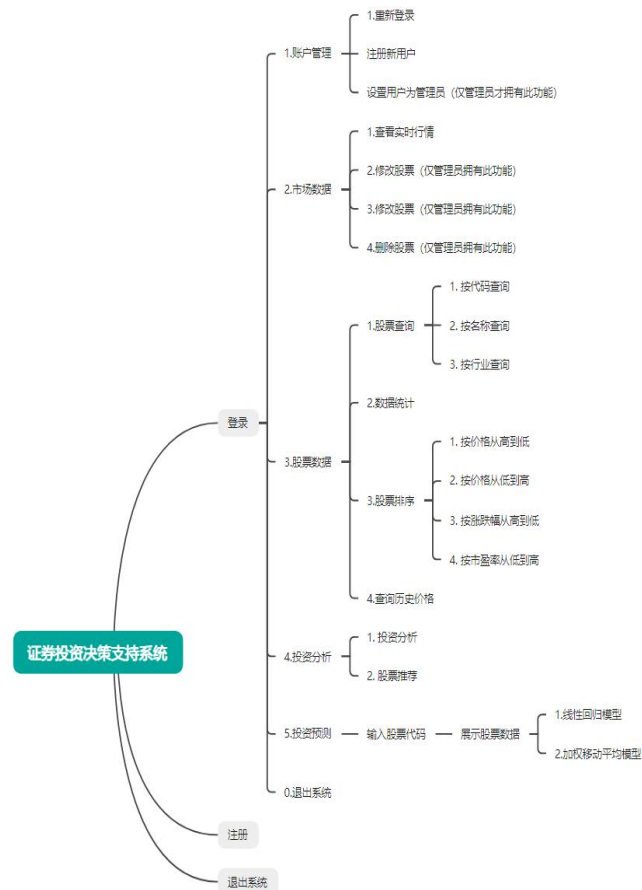
编写一个证券投资决策支持系统，能够实现对股票数据的查询，预测等功能，同时拥有账号的登录注册等。

基本功能：

1. 用户登录系统：在初始界面需要输入正确的用户名和对应的密码才能进入系统
2. 查看市场行情：股票名称，代码，近十天的价格，市盈率，涨跌幅，所属行业等
3. 添加，修改，删除股票数据：管理员可以对股票数据进行操作
4. 查询股票：通过股票名称，代码，所属行业对股票进行查询
5. 投资分析：对所有股票的数据进行整体分析
6. 股票推荐：借助市盈率，涨跌幅等数据对股票进行智能推荐
7. 投资预测：可以选择使用线性回归模型或者加权移动平均模型对股票价格进行预测
8. 历史数据保存功能：对保存的用户数据和股票数据进行保存
9. 管理员权限功能：管理员可以赋予普通用户管理员功能

2. 软件整体规划及设计

在给课程设计刚发布时，因为要求使用图形化界面，我曾想过使用 Qt 等工具进行编程，但在学习了一段事件之后，发现学习量巨大但时间不足，于是放弃，只好使用控制台进行简单的操作。



1. 流程图:

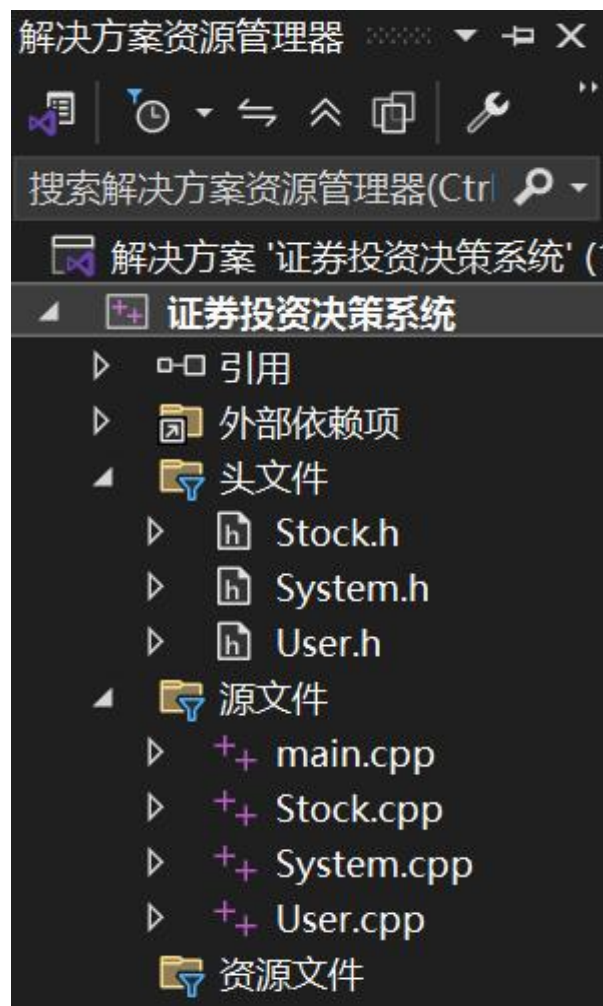
2.类的设计

类的设计中，我分为了三个类：

- (1) 股票类 (Stock.h)
- (2) 用户类 (User.h)
- (3) 系统类 (System.h)

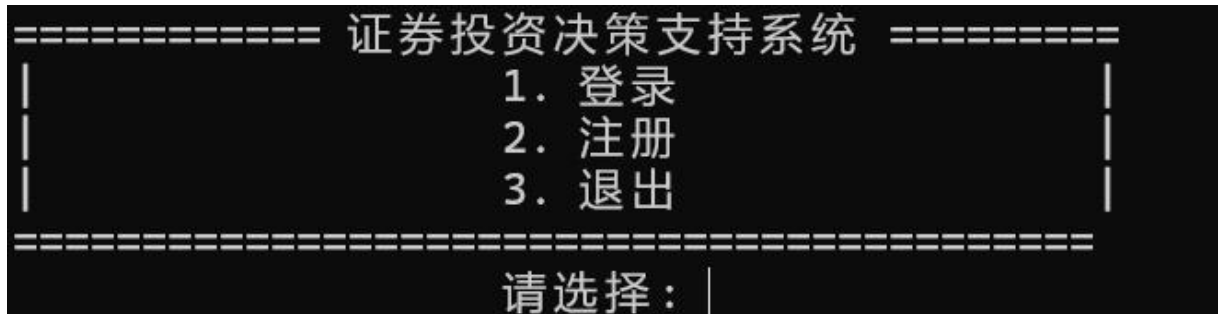
其中，股票类存储股票的基本信息，计算股票的涨跌幅等，并可以输出信息到文件和从文件读取信息。用户类课拥有存储用户数据，从从文件读取信息，输出信息到文件。系统类用于绘制菜单界面，灵活调用其他两个类的函数。

程序结果如图所示：



3. 程序详细设计

一. 登录界面:

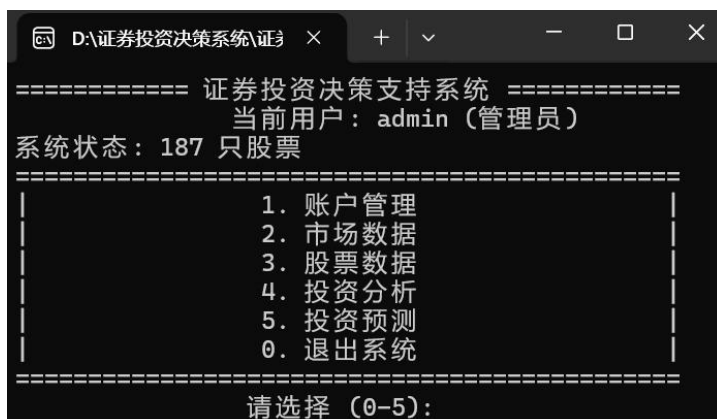


输入正确的用户名和密码即可登录。信息保存在 user.txt 中

其中，最后一个数字的为管理员



在正确输入或注册之后，即可进入系统：



当前为管理员身份进入。

接下来时代码解析：

登录界面通过一下代码判断是否登陆成功：

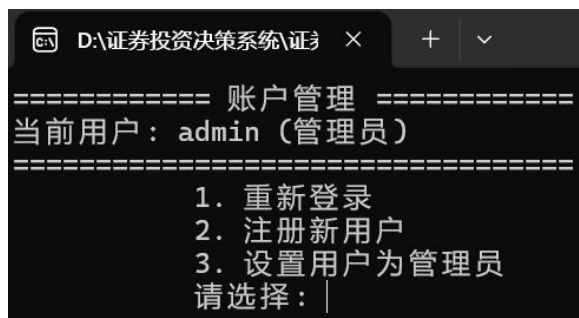
```
// 登录操作
void System::login()
{
    clearScreen();
    cout << "===== 用户登录 =====\n";
    string username, password;
    cout << "                用户名: ";
    cin >> username;
    cout << "                密码: ";
    cin >> password;
    for (auto &user : users)
    {
        if (user.getUsername() == username && user.checkPassword(password))
        {
            currentUser = user;
            cout << "\n        登录成功! 欢迎 " << username << endl;
            waitForKey();
            return;
        }
    }
    cout << "\n 用户名或密码错误!" << endl;
    waitForKey();
}
```

users 是一个储存用户数据的一个容器，在 void System::loadUsers() 函数中创建。通过遍历容器判断是否存在。如果存在，即可进入 run() 函数的下一级循环。

二. 账户管理功能

在主菜单中输入 1 并按回车进入账户管理功能界面

如图：



仅管理员可以选择此选项

```
else if (choice == 3 && currentUser.getIsAdmin())
{
    setUserAsAdmin();
}

// 检查当前用户是否为管理员
if (!currentUser.getIsAdmin())
{
    cout << "只有管理员可以设置其他账户为管理员! " << endl;
    waitForKey();
    return;
}

// 显示当前所有用户
cout << "当前系统用户列表:\n";
cout << "-----\n";
cout << "|用户名\t\t\t\t\t状态| \n";
cout << "-----\n";
for (const auto &user : users)
{
    cout << user.getUsername();
    if (user.getUsername().length() < 8)
        cout << "\t\t\t";
    else
        cout << "\t";
    if (user.getIsAdmin())
        cout << "管理员" << endl;
    else
        cout << "普通用户" << endl;
}
cout << "-----\n";

// 输入要设置的用户名
string username;
cout << "\n 请输入要设置为管理员的用户名: ";
cin >> username;
```

首先展示所有现存用户，并显示当前身份，再输入需要修改的用户名


```

// 查找用户
bool found = false;
for (auto &user : users)
{
    if (user.getUsername() == username)
    {
        found = true;
        // 检查是否已经是管理员
        if (user.getIsAdmin())
        {
            cout << "\n 用户 " << username << " 已经是管理员! " << endl;
            waitForKey();
            return;
        }
        // 验证管理员密码
        string adminPassword;
        cout << "\n 需要验证管理员权限!" << endl;
        cout << "请输入当前管理员密码进行验证: ";
        cin >> adminPassword;
        // 验证密码
        if (!currentUser.checkPassword(adminPassword))
        {
            cout << "\n 密码错误! 权限验证失败!" << endl;
            waitForKey();
            return;
        }
        // 确认操作
        cout << "\n 密码验证通过!" << endl;
        cout << "\n 确定要将用户 \"" << username << "\" 设置为管理员吗? (y/n): ";
        char confirm;
        cin >> confirm;
    }
}

```

当要修改时，必须再次输入管理员的密码才可修改身份

三. 市场数据

(1) 展示市场数据

```
void System::showAllStocks()
{
    clearScreen();
    cout << "----- 实时行情 -----\n";
    if (stocks.empty())
    {
        cout << "股票列表为空!" << endl;
        cout << "请先添加股票数据或确保 stocks.txt 文件存在且格式正确。" << endl;
        waitForKey();
        return;
    }
    // 标题栏: 调整列宽, 确保对齐
    // 总宽度: 8+16+12+10+10+12+12 = 80 字符
    cout << setw(8) << left << "代码"
        << setw(16) << left << "名称"
        << setw(12) << right << "当前价格"
        << setw(10) << right << "涨跌幅"
        << setw(10) << right << "市盈率"
        << setw(12) << right << "行业"
        << setw(12) << right << "成交量" << endl; // 保持 12

    // 分隔线: 用 "-" 填充, 总宽度 80 字符
    cout << string(80, '-') << endl;

    // 循环显示股票数据
    for (auto &stock : stocks)
    {
        stock.display();
    }

    // 底部统计信息
    cout << string(80, '-') << endl;
    cout << "总计: " << stocks.size() << " 只股票" << endl;
    waitForKey();
}
```

以下是展示所有股票数据的函数 `display()`

```
void Stock::display() const
{
    // 修改列宽
```

```

cout << setw(8) << left << code; // 股票代码
cout << setw(16) << left << name; // 股票名称
cout << setw(12) << right << fixed << setprecision(2) << price; // 当前价格
// 涨跌幅：调整为 10 字符宽，包含正负号和百分号
cout << setw(10) << right;
if (change > 0)
{
    //ANSI 颜色代码
    cout << "\033[31m+" << fixed << setprecision(2) << change << "%\033[0m"; // 红色上涨
}
else if (change < 0)
{
    cout << "\033[32m" << fixed << setprecision(2) << change << "%\033[0m"; // 绿色下跌
}
else
{
    cout << fixed << setprecision(2) << change << "%"; // 平盘，前面加空格对齐
}

// 市盈率：调整为 10 字符宽
cout << setw(10) << right << fixed << setprecision(2) << peRatio;

// 行业：调整为 12 字符宽，左对齐
cout << setw(12) << right << industry;

// 成交量：调整为 12 字符宽，右对齐
cout << setw(12) << right << volume;

cout << endl;
}

ofstream& operator<<(ofstream& fout, const Stock& stock)
{
    fout << stock.code << " ";
    fout << stock.name << " ";
    fout << stock.price << " ";
    fout << stock.change << " ";
    fout << stock.peRatio << " ";
    fout << stock.industry << " ";
    fout << stock.volume << " ";

    // 保存最近 10 天历史价格（每行末尾 10 个数字）
    const int WANT = 10;
    int h = static_cast<int>(stock.historyPrices.size());
    double pad = stock.price;

```

```

// 如果不足 10 个，用当前价格填充到 10 个
if (h >= WANT)
{
    for (int i = h - WANT; i < h; ++i)
        fout << stock.historyPrices[i] << " ";
}
else
{
    for (int i = 0; i < WANT - h; ++i)
        fout << pad << " ";
    for (int i = 0; i < h; ++i)
        fout << stock.historyPrices[i] << " ";
}

return fout;
}

```

```

if (change > 0)
{
    //ANSI 颜色代码
    cout << "\033[31m+" << fixed << setprecision(2) << change << "%\033[0m"; // 红色上涨
}

```

这一段用于设置股票数据的颜色，如果涨跌幅大于 0，使用红色显示，如果涨跌幅小于 0，使用绿色显示。

(2) 添加股票功能：

```

void System::addStock()
{
    clearScreen();
    if (!currentUser.getIsAdmin())
    {
        cout << "只有管理员可以添加股票！" << endl;
        waitForKey();
        return;
    }
    cout << "===== 添加股票 =====\n";

    string code, name, industry;
    double price, pe;
    cout << "股票代码： ";
    cin >> code;
    if (findStockByCode(code))

```

```

{
    cout << "股票代码已存在！" << endl;
    waitForKey();
    return;
}
cout << "股票名称: ";
cin >> name;
cout << "当前价格: ";
cin >> price;
cout << "市盈率: ";
cin >> pe;
cout << "所属行业: ";
cin >> industry;
// 要求输入最近 10 天价格
vector<double> prices;
prices.reserve(10);
cout << "请输入最近 10 天价格(按时间顺序, 最早到最近), 用空格或回车分隔:\n";
for (int i = 0; i < 10; ++i)
{
    double p;
    while (!(cin >> p))
    {
        cin.clear();
        string bad;
        cin >> bad;
        cout << "输入无效, 请输入数字价格: ";
    }
    prices.push_back(p);
}
Stock s(code, name, price, pe, industry);
s.setHistoryPrices(prices);
stocks.push_back(s);
saveStocks();
cout << "\n 股票添加成功!" << endl;
waitForKey();
}

```

首先判断是不是管理员因为只有管理员才可以添加股票,

```

if (findStockByCode(code))
{
    cout << "股票代码已存在！" << endl;
    waitForKey();
    return;
}

```

再对输入的股票代码进行判断，遍历储存股票数据的容器，如果发现已经存在相同代码的股票，则结束进程。按照提示输入股票相关信息之后，股票数据会被储存在 stocks.txt 文件中。（调用 saveStocks() 函数）

对于修改股票数据，不过是对原有股票数据的覆盖，这里不多赘述。

四．股票数据

（1）股票查询功能

股票可以通过代码，名称，行业进行查询，下面以行业查询为例进行介绍。

```
cout << "请输入行业名称: ";
cin >> keyword;
for (auto &stock : stocks)
{
    if (stock.getIndustry() == keyword)
    {
        results.push_back(stock);
    }
}
```

在输入行业名称之后，对储存股票数据的容器进行遍历，如过查询到目标股票，则将这个股票储存在 result 这个容器中。

```
cout << "===== 查询结果 =====\n";
if (results.empty())
{
    cout << "未找到符合条件的股票。" << endl;
}
else
{
    cout << "找到 " << results.size() << " 只符合条件的股票: \n";
    cout << "代码      名称                当前价格    涨跌幅    市盈率      行业      成交量\n";
    cout <<
    "-----\n";
    for (auto &stock : results)
    {
        stock.display();
    }
}
```

```

    }
    cout <<
    "-----\n";
}

    waitForKey();

```

通过遍历 result 这个容器，将查询到的结果显示。

(2) 数据统计

```

void System::statistics() // 行业统计
{
    clearScreen();
    cout << "===== 数据统计 =====\n";

    if (stocks.empty())
    {
        cout << "股票列表为空，无法统计！" << endl;
        waitForKey();
        return;
    }
    // 统计股票数量
    cout << "股票总数：" << stocks.size() << endl;
    // 统计各行业股票数量
    cout << "\n 行业分布：" << endl;
    cout << "行业      股票种类数量\n";
    cout << "-----\n";
    vector<string> industries;
    for (auto &stock : stocks)
    {
        industries.push_back(stock.getIndustry());
    }
    // 去重
    sort(industries.begin(), industries.end());
    industries.erase(unique(industries.begin(), industries.end()), industries.end());

    // 统计并显示
    for (auto &industry : industries)
    {
        int count = 0;
        for (auto &stock : stocks)
        {

```

```

        if (stock.getIndustry() == industry)
            count++;
    }
    cout << setw(10) << left << industry;
    cout << setw(4) << right << count << endl;
}

// 价格区间统计
cout << "\n 价格区间分布:" << endl;
int low = 0, mid = 0, high = 0;
for (auto &stock : stocks)
{
    double price = stock.getPrice();
    if (price < 10)
        low++;
    else if (price < 100)
        mid++;
    else
        high++;
}

// 使用图标显示
cout << "0-10 元:  ";
for (int i = 0; i < low; i++)
    cout << "■";
cout << " (" << low << ")\n";
cout << "10-100 元: ";
for (int i = 0; i < mid; i++)
    cout << "■";
cout << " (" << mid << ")\n";
cout << "100 元以上:";
for (int i = 0; i < high; i++)
    cout << "■";
cout << " (" << high << ")\n";
waitForKey();
}

```

首先遍历所有股票，对所有的行业进行统计，再使用 `erase` 进行去重，然后保留不同的行业，再对每个行业的股票数量进行统计。同时进行价格区间统计。

(3) 股票排序

```
void System::sortStocks()
```



```

{
    clearScreen();
    cout << "===== 股票排序 =====\n";
    if (stocks.empty())
    {
        cout << "股票列表为空，无法排序！" << endl;
        waitForKey();
        return;
    }
    cout << "    1. 按价格从高到低\n";
    cout << "    2. 按价格从低到高\n";
    cout << "    3. 按涨跌幅从高到低\n";
    cout << "    4. 按市盈率从低到高\n";
    cout << "    请选择： ";
    int choice;
    cin >> choice;
    vector<Stock> sortedStocks = stocks;

    switch (choice)
    {
    case 1:
        sort(sortedStocks.begin(), sortedStocks.end(),
            [](const Stock &a, const Stock &b)
            { return a.getPrice() > b.getPrice(); });
        break;
    case 2:
        sort(sortedStocks.begin(), sortedStocks.end(),
            [](const Stock &a, const Stock &b)
            { return a.getPrice() < b.getPrice(); });
        break;
    case 3:
        sort(sortedStocks.begin(), sortedStocks.end(),
            [](const Stock &a, const Stock &b)
            { return a.getChange() > b.getChange(); });
        break;
    case 4:
        sort(sortedStocks.begin(), sortedStocks.end(),
            [](const Stock &a, const Stock &b)
            { return a.getPERatio() < b.getPERatio(); });
        break;
    default:
        cout << "无效选择！" << endl;
        waitForKey();
        return;
    }
}

```

```

    }
    cout << "\n 排序结果:" << endl;
    cout << "代码      名称      当前价格      涨跌幅      市盈率      行业      成交量
\n";
    cout <<
    "-----\n";
    for (auto &stock : sortedStocks)
    {
        stock.display();
    }
    waitForKey();
}

```

使用<algorithm>库中的 sort() 排序函数进行排序

五. 投资分析

(1) 投资分析功能

```

void System::analysis()
{
    clearScreen();
    cout << "===== 投资分析 =====\n";

    if (stocks.empty())
    {
        cout << "股票列表为空，无法分析！" << endl;
        waitForKey();
        return;
    }

    // 市盈率分析
    cout << "市盈率分析:" << endl;
    int lowPE = 0, normalPE = 0, highPE = 0;

    for (auto &stock : stocks)
    {
        double pe = stock.getPERatio();
        if (pe < 15)
            lowPE++;
    }
}

```

```

        else if (pe < 30)
            normalPE++;
        else
            highPE++;
    }

    cout << "低市盈率(<15): " << lowPE << " 只\n";
    cout << "中市盈率(15-30):" << normalPE << " 只\n";
    cout << "高市盈率(>30): " << highPE << " 只\n";

    // 涨跌幅分析
    cout << "\n 涨跌幅分析:" << endl;
    int rise = 0, fall = 0, flat = 0;

    for (auto &stock : stocks)
    {
        double change = stock.getChange();
        if (change > 0)
            rise++;
        else if (change < 0)
            fall++;
        else
            flat++;
    }

    cout << "上涨: " << rise << " 只\n";
    cout << "下跌: " << fall << " 只\n";
    cout << "平盘: " << flat << " 只\n";

    waitForKey();
}

```

对不同市盈率的股票进行统计。

(2) 股票推荐

```

void System::recommendation()
{
    clearScreen();
    cout << "===== 股票推荐
=====\\n";
}

```

```

    if (stocks.empty())
    {
        cout << "                        股票列表为空，无法推荐！" << endl;
        waitForKey();
        return;
    }
    // 推荐低市盈率股票
    cout << "低市盈率推荐（价值投资）：" << endl;
    cout << "代码          名称          当前价格      涨跌幅      市盈率      行业      成交量
\n";
    cout <<
    "-----\n";

    int count = 0;
    for (auto &stock : stocks)
    {
        if (stock.getPERatio() < 20 && stock.getPERatio() > 0)
        {
            stock.display();
            count++;
            if (count >= 15)
                break; // 只显示前 15 个
        }
    }
    if (count == 0)
    {
        cout << "暂无符合条件的股票。" << endl;
    }
    waitForKey();
}

```

依据股票的市盈率对股票进行推荐，如果市盈率大于 0 小于 20 则推荐。

六. 投资预测

```

// ===== 投资预测 =====
void System::prediction()
{
    clearScreen();
    cout << "===== 投资预测 =====\n";
}

```

```

if (stocks.empty())
{
    cout << "股票列表为空，无法预测！" << endl;
    waitForKey();
    return;
}
string code;
cout << "请输入股票代码：";
cin >> code;
Stock *stock = findStockByCode(code);
if (!stock)
{
    cout << "股票不存在！" << endl;
    waitForKey();
    return;
}
cout << "\n 股票： " << stock->getName() << " (" << code << ")" << endl;
cout << "当前价格： " << fixed << setprecision(2) << stock->getPrice() << endl;
// 获取历史价格
vector<double> prices = stock->getHistoryPrices();
int historySize = static_cast<int>(prices.size());
if (historySize < 5)
{
    cout << "\n 历史数据不足（当前仅有 " << historySize << " 天数据）" << endl;
    cout << "至少需要 5 天历史数据才能进行有效预测。" << endl;
    cout << "\n 是否手动输入最近 10 天价格进行预测？(y/n)： ";
    char choice;
    cin >> choice;
    if (choice == 'y' || choice == 'Y')
    {
        prices.clear();
        cout << "\n 请输入最近 10 天价格（按时间顺序，最早到最近）：" << endl;
        for (int i = 0; i < 10; ++i)
        {
            cout << "第" << (i + 1) << "天价格： ";
            double p;
            while (!(cin >> p))
            {
                cin.clear();
                string bad;
                cin >> bad;
                cout << "输入无效，请输入数字价格： ";
            }

```

```

        prices.push_back(p);
    }
    historySize = 10;
}
else
{
    waitForKey();
    return;
}
}

// 显示最近价格
cout << "\n 最近" << min(10, historySize) << "天价格: ";
int startIdx = historySize - min(10, historySize);
for (int i = startIdx; i < historySize; ++i)
{
    cout << fixed << setprecision(2) << prices[i] << " ";
}
cout << endl;
// 选择预测模型
cout << "\n===== 选择预测模型 =====\n";
cout << "          1. 线性回归模型\n";
cout << "          2. 加权移动平均模型\n";
cout << "          请选择 (1/2): ";
int modelChoice;
cin >> modelChoice;
if (modelChoice != 1 && modelChoice != 2)
{
    cout << "使用默认模型 (线性回归)" << endl;
    modelChoice = 1;
}
// 计算预测结果
vector<double> predictions;
if (modelChoice == 1)
{
    cout << "\n 使用线性回归模型预测..." << endl;
    predictions = predictLinear(prices);
}
else
{
    cout << "\n 使用加权移动平均模型预测..." << endl;
    predictions = predictWMA(prices);
}

```

```

double currentPrice = stock->getPrice();
// 显示预测结果
cout << "\n===== 预测结果 =====\n";
cout << "当前价格: " << fixed << setprecision(2) << currentPrice << endl;
cout << "-----\n";
// 显示三天预测
string dayNames[] = {"明天", "后天", "大后天"};
for (int i = 0; i < 3; ++i)
{
    double predPrice = predictions[i];
    double changeRate = ((predPrice - currentPrice) / currentPrice) * 100.0;
    cout << dayNames[i] << "预测: ";
    cout << fixed << setprecision(2) << predPrice;
    cout << " (";
    if (changeRate > 0)
    {
        cout << "+" << fixed << setprecision(2) << changeRate << "%)";
    }
    else if (changeRate < 0)
    {
        cout << fixed << setprecision(2) << changeRate << "%)";
    }
    else
    {
        cout << "0.00%)";
    }
    cout << endl;
}
// 简单投资建议
cout << "\n===== 投资建议 =====\n";
// 基于预测结果给出建议
double avgChange = 0.0;
for (int i = 0; i < 3; ++i)
{
    avgChange += ((predictions[i] - currentPrice) / currentPrice) * 100.0;
}
avgChange /= 3.0;
if (avgChange > 2.0)
{
    cout << "预测整体上涨趋势明显, 可考虑关注." << endl;
}
else if (avgChange > 0)
{
    cout << "预测小幅上涨, 建议谨慎观望." << endl;
}

```

```

    }
    else if (avgChange > -2.0)
    {
        cout << "预测变化不大，建议保持观望。" << endl;
    }
    else
    {
        cout << "预测呈下跌趋势，建议注意风险。" << endl;
    }
    // 基于市盈率的建议
    double peRatio = stock->getPERatio();
    if (peRatio > 0)
    {
        cout << "\n 市盈率分析：" << endl;
        if (peRatio < 15)
            cout << "估值较低，具备投资价值" << endl;
        else if (peRatio < 30)
            cout << "估值合理" << endl;
        else if (peRatio < 50)
            cout << "估值偏高，需谨慎" << endl;
        else
            cout << "估值过高，注意风险" << endl;
    }
    cout << "\n 提示：预测仅供参考，不构成投资建议!\n 理财有风险，投资需谨慎！！!" << endl;
    cout << "=====\n";
    waitForKey();
}

```

首先要求输入股票代码，

再遍历 stock 容器查找股票数据，如果成功找到则显示此股票数据。

程序提供了两种预测模型，分别是基于最小二乘法的线性回归模型，还有一种是加权移动平均模型。通过模型计算出之后三天的预测价格得出涨跌幅，根据涨跌幅的范围给出相应的建议。

下面分别来解释这两种模型：

1. 线性回归模型：


```

// 线性回归预测未来三天价格
vector<double> System::predictLinear(const vector<double> &prices)
{
    vector<double> predictions;
    int n = static_cast<int>(prices.size()); // 获取总天数

    if (n < 2)
    {
        // 数据不足，用最后一天价格填充
        double lastPrice = prices.empty() ? 0.0 : prices.back();
        predictions.push_back(lastPrice);
        predictions.push_back(lastPrice);
        predictions.push_back(lastPrice);
        return predictions;
    }

    // 线性回归计算  $b = (\text{sumXY} - n * \text{meanX} * \text{meanY}) / (\text{sumXX} - n * \text{meanX} * \text{meanX})$ 
    double sumX = 0, sumY = 0, sumXY = 0, sumXX = 0;
    for (int i = 0; i < n; ++i)
    {
        double x = static_cast<double>(i); // 时间变量
        double y = prices[i]; // 时间对应的价格变量
        sumX += x; // x 的和
        sumY += y; // y 的和
        sumXY += x * y; // xy 的和
        sumXX += x * x; // x?的和
    }

    double meanX = sumX / n; // x 的平均数
    double meanY = sumY / n; // y 的平均数
    double b = sumXX - n * meanX * meanX; // 斜率 b 的值

    double slope = 0.0;
    if (fabs(b) > 1e-12)
    {
        slope = (sumXY - n * meanX * meanY) / b;
    }

    double a = meanY - slope * meanX; // 截距 a 的值

    // 预测未来三天
    for (int i = 1; i <= 3; ++i)
    {
        double nextX = static_cast<double>(n - 1 + i);
        double predicted = a + slope * nextX;
    }
}

```

```

        predictions.push_back(predicted);
    }
    return predictions;
}

```

根据公式： $b = \frac{\sum [(x_i - x)(y_i - y)]}{\sum [(x_i - x)]^2}$

$a = y - b * x$;

得出最小二乘法的公式，再预测之后三天的价格。

2. 加权移动平均模型

// 加权移动平均预测未来三天价格（近期的数据权重比较大，趋势影响更大）

```

vector<double> System::predictWMA(const vector<double> &prices)
{
    // 假设已经有至少 10 天数据
    int n = static_cast<int>(prices.size());

    if (n < 10)
    {
        // 万一数据不够，保守处理
        double lastPrice = prices.empty() ? 0.0 : prices.back();
        return vector<double>(3, lastPrice);
    }

    // 使用最近 10 天数据计算加权平均
    double weightedSum = 0.0;
    double weightSum = 0.0;

    // 权重：最近的天数权重高 (1, 2, 3, ..., 10)
    for (int i = n - 10; i < n; ++i)
    {
        double weight = (i - (n - 10) + 1); // 权重从 1 到 10
        weightedSum += prices[i] * weight; // 加权价格和
        weightSum += weight; // 权重和
    }

    double basePrediction = weightedSum / weightSum; // WMA 值

    // 以 WMA 为基准价格，下面使用最近三天的数据计算趋势，进行调整预测价格。
}

```

```

// 计算趋势：用最近 3 天的平均变化（比只用最后一天更稳定）
double recentTrend = 0.0;
if (n >= 3)
{
    // 最近 3 天的平均日变化
    double change1 = prices[n - 1] - prices[n - 2]; // 昨天的和前天的
    double change2 = prices[n - 2] - prices[n - 3]; // 前天的和大前天的
    recentTrend = (change1 + change2) / 2.0; // 平均每天变化
}

// 预测未来三天
vector<double> predictions;
// 预测价格=基准价格+趋势*衰减因子*天数
for (int day = 1; day <= 3; ++day)
{
    // 趋势衰减：预测越远，趋势影响越小
    double decayFactor = 1.0 / (1.0 + day * 0.3); // 衰减因子
    double predicted = basePrediction + recentTrend * decayFactor * day;

    predictions.push_back(predicted);
}

return predictions;
}

// 计算涨跌幅百分比

double System::calculateChangeRate(double currentPrice, double predictedPrice)
{
    if (fabs(currentPrice) < 1e-12)
        return 0.0;
    return ((predictedPrice - currentPrice) / currentPrice) * 100.0;
}

```

根据当前的时间，为每个数据设置权重，其中前一天的权重是 10，而十天前的数据权重设置为 1，这样可以反映每个数据的重要程度，计算出每个数据乘以权重的和，再除以权重总和得出一个基准数据，再通过这个基准，算出未来三天的价格：

预测价格=基准价格+趋势*衰减因子*天数

七. 退出系统

```
case 0: // 退出
    cout << "感谢使用, 再见!" << endl;
    return;
```

八. 其他重要函数

(1) 重载运算符<< (输出股票信息到文件)

```
ostream& operator<<(ostream& fout, const Stock& stock)
{
    fout << stock.code << " ";
    fout << stock.name << " ";
    fout << stock.price << " ";
    fout << stock.change << " ";
    fout << stock.peRatio << " ";
    fout << stock.industry << " ";
    fout << stock.volume << " ";

    // 保存最近 10 天历史价格 (每行末尾 10 个数字)
    const int WANT = 10;
    int h = static_cast<int>(stock.historyPrices.size());
    double pad = stock.price;
    // 如果不足 10 个, 用当前价格填充到 10 个
    if (h >= WANT)
    {
        for (int i = h - WANT; i < h; ++i)
            fout << stock.historyPrices[i] << " ";
    }
    else
    {
        for (int i = 0; i < WANT - h; ++i)
            fout << pad << " ";
        for (int i = 0; i < h; ++i)
            fout << stock.historyPrices[i] << " ";
    }

    return fout;
}
```

Ofstream& : 返回输出文件流的引用, 支持链式调用

operator<< : 重载输出运算符

Fout: 输出文件流对象

const Stock& stock: 常量引用传递的 Stock 对象

输出信息示例:

600519 贵州茅台 1800.5 0.13 40.2 食品 1850000 1780.25 1785 1792.75 1795.6 1800
1802.1 1805.3 1798.2 1800.5 1803.8

(2) 重载运算符>> (从文件获取股票信息)

```
ifstream& operator>>(ifstream& fin, Stock& stock)
{
    // 读取整行并解析: 最后 10 个数字视为最近 10 天价格
    std::string line;
    if (!std::getline(fin, line))
        return fin;

    std::istringstream iss(line);
    iss >> stock.code;
    iss >> stock.name;
    iss >> stock.price;
    iss >> stock.change;
    iss >> stock.peRatio;
    iss >> stock.industry;
    iss >> stock.volume;

    // 读取余下所有数字作为历史价格候选
    std::vector<double> allPrices;
    double p;
    while (iss >> p)
    {
        allPrices.push_back(p);
    }

    stock.historyPrices.clear();
    const int WANT = 10;
    int m = static_cast<int>(allPrices.size());
```

```

    if (m >= WANT)
    {
        for (int i = m - WANT; i < m; ++i)
            stock.historyPrices.push_back(allPrices[i]);
    }
    else
    {
        // 如果不足 10 个，则用当前价格前置填充
        for (int i = 0; i < WANT - m; ++i)
            stock.historyPrices.push_back(stock.price);
        for (int i = 0; i < m; ++i)
            stock.historyPrices.push_back(allPrices[i]);
    }

    return fin;
}

```

ifstream&: 返回输入文件流的引用，支持链式调用

Operator>>: 重载输入运算符

fin: 输入文件流对象

功能: 从文件中读取一行数据，解析并填充到 Stock 对象中

(3) 重载运算符<< (输出用户信息到文件)

```

ofstream& operator<<(ofstream& fout, const User& user) {
    fout << user.username << " ";
    fout << user.password << " ";
    fout << (user.isAdmin ? 1 : 0);
    return fout;
}

```

输出示例: admin admin123 1

分别为: 用户名 密码 管理员的标志位 (1 为管理员)

(4) 重载运算符>> (从文件读取用户信息)

```
ifstream& operator>>(ifstream& fin, User& user) {  
    int adminFlag;  
    fin >> user.username;  
    fin >> user.password;  
    fin >> adminFlag;  
    user.isAdmin = (adminFlag == 1);  
    return fin;  
}
```

(5) 清屏函数

```
void System::clearScreen()  
{  
#ifdef _WIN32  
    system("cls");  
#else  
    system("clear");  
#endif  
}
```

用于清楚屏幕中已存在的文字等信息。

(6) System 类的构造函数

```
System::System()  
{  
    loadUsers();  
    loadStocks();  
  
    // 如果没有用户, 创建默认管理员  
    if (users.empty())  
    {  
        users.push_back(User("admin", "admin123", true));  
        saveUsers();  
    }  
}
```

如果 User.tet 文件中没有用户数据, 则创建一个默认的管理员用户。

(7)

加载股票数据

```
void System::loadStocks()
{
    ifstream fin("stocks.txt");
    if (!fin)
    {
        cout << "注意：找不到 stocks.txt 文件！" << endl;
        cout << "请创建 stocks.txt 文件并添加股票数据。" << endl;
        cout << "格式：代码 名称 当前价格 涨跌幅 市盈率 行业 成交量 最近 10 天价格(10 个数字)" <<
endl;
        cout << "示例：600519 贵州茅台 1800.50 0.00 40.20 白酒 1850000 1780.25 1785.00 1792.75
1795.60 1800.00 1802.10 1805.30 1798.20 1800.50 1803.80" << endl;
        waitForKey();
        return; // 不初始化任何数据，直接返回
    }

    // 清空现有股票列表
    stocks.clear();

    Stock stock;
    int count = 0;
    while (fin >> stock)
    {
        stocks.push_back(stock);
        count++;
    }
    fin.close();

    if (count == 0)
    {
        cout << "错误：stocks.txt 文件为空或格式不正确！" << endl;
        cout << "请确保文件包含至少一只股票的数据。" << endl;
        waitForKey();
    }
    else
    {
        cout << "系统初始化完成，一共存在 " << count << " 只股票。" << endl;
    }
}
```

读取文件夹中的 stocks.txt 文件，如果没有成功读取，则提示用户输入正

确的数据。

（7）保存股票数据

```
void System::saveStocks()
{
    ofstream fout("stocks.txt");
    if (!fout)
    {
        cout << "错误：无法保存股票数据到文件！" << endl;
        return;
    }

    for (auto &stock : stocks)
    {
        fout << stock << endl;
    }
    fout.close();

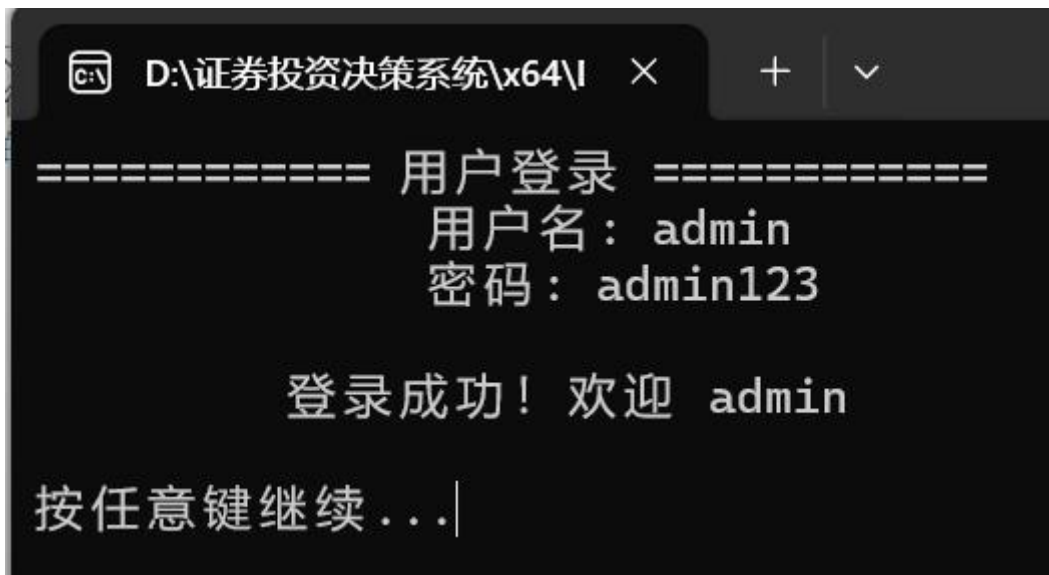
    cout << "股票数据已保存到 stocks.txt" << endl;
}

Stock *System::findStockByCode(string code)
{
    for (auto &stock : stocks)
    {
        if (stock.getCode() == code)
        {
            return &stock;
        }
    }
    return nullptr;
}
```

首先尝试打开 stocks.txt 文件，成功打开之后再文件的最后添加最新的股票数据。

4. 程序测试

(1) 登录系统



```
D:\证券投资决策系统\x64\I >

===== 用户登录 =====
      用户名： admin
      密码： admin123

      登录成功！ 欢迎 admin

按任意键继续...|
```

(2) 设置用户为管理员

```
===== 设置管理员账户 =====
当前系统用户列表：
-----
|用户名                | 状态
|
-----
admin                管理员
qwe                  管理员
123                  普通用户
abc                  普通用户
-----

请输入要设置为管理员的用户名： 123

需要验证管理员权限！
请输入当前管理员密码进行验证： admin123

密码验证通过！

确定要将用户 "123" 设置为管理员吗？ (y/n): y

用户 123 已成功设置为管理员！

-----
用户名                状态
-----
123                    管理员
-----

按任意键继续...|
```

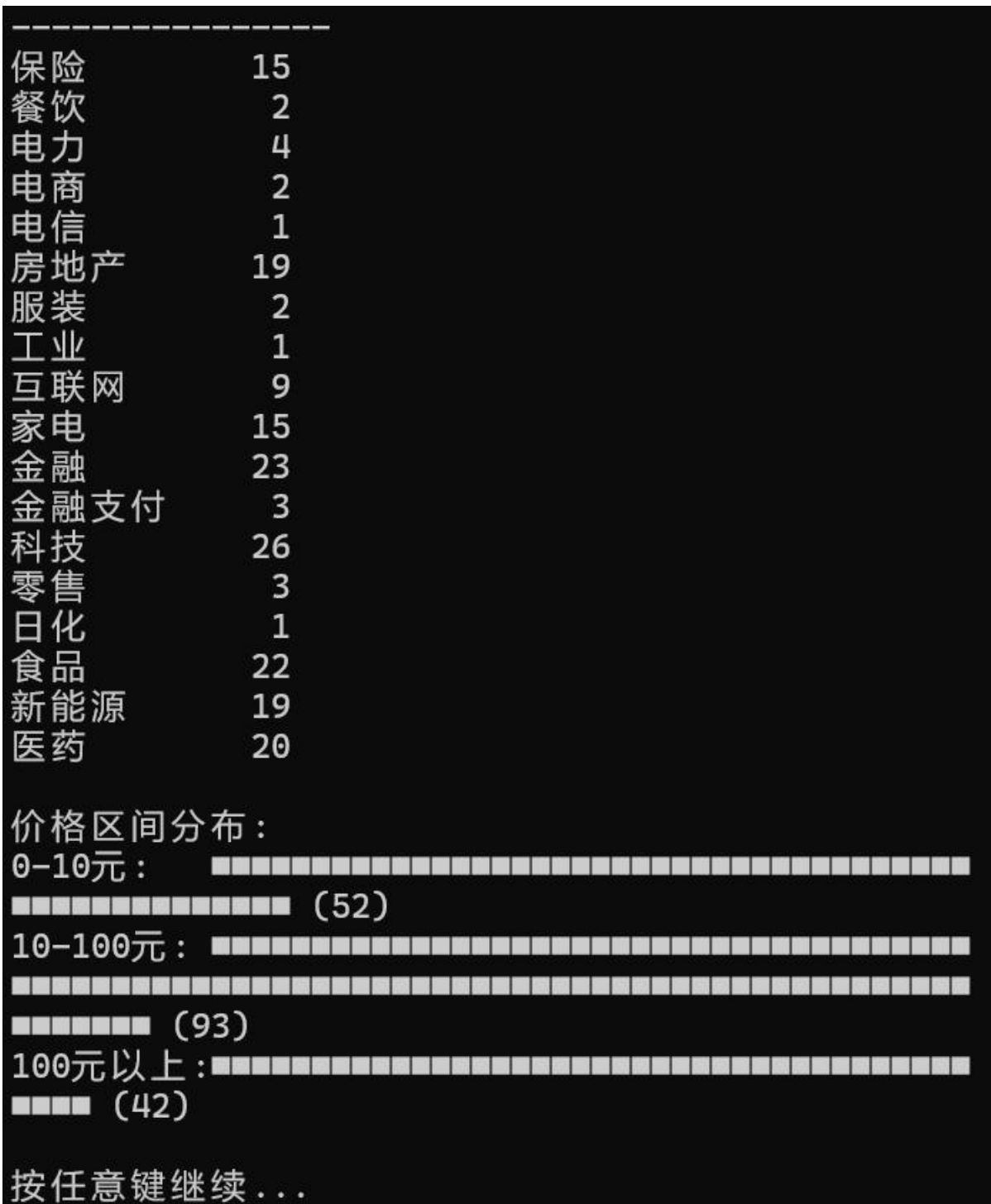
(3) 查看当前行情

600518	康美药业	2.85	+0.35%
35.80	医药	456000000	
000031	大悦城	4.25	+0.47%
12.50	房地产	345000000	
000402	金融街	5.68	+0.35%
8.90	房地产	456000000	
000069	华侨城A	4.85	+0.41%
15.80	房地产	567000000	
000014	沙河股份	10.25	+0.68%
22.50	房地产	156000000	
000029	深深房A	7.85	+0.51%
18.90	房地产	234000000	
000036	华联控股	4.25	+0.47%
15.80	房地产	345000000	
000040	东旭蓝天	2.85	+0.35%
28.50	房地产	456000000	
000046	泛海控股	1.25	+0.40%
35.80	房地产	567000000	
000056	皇庭国际	2.45	+0.41%
45.80	房地产	456000000	
000090	天健集团	5.25	+0.48%
12.50	房地产	234000000	
000090	天健集团	5.25	+0.48%
12.50	房地产	234000000	

总计：187 只股票

按任意键继续...

(4) 股票数据统计



(5) 投资预测

```
===== 投资预测 =====
请输入股票代码：000002

股票：万科A (000002)
当前价格：9.85

最近10天价格：9.72 9.75 9.78 9.80 9.82 9.85 9.83
               9.84 9.85 9.86

===== 选择预测模型 =====
                1. 线性回归模型
                2. 加权移动平均模型
                请选择 (1/2): 2

使用加权移动平均模型预测...

===== 预测结果 =====
当前价格：9.85
-----
明天预测：9.84 (-0.11%)
后天预测：9.84 (-0.06%)
大后天预测：9.85 (-0.03%)

===== 投资建议 =====
预测变化不大，建议保持观望。

市盈率分析：
估值较低，具备投资价值

提示：预测仅供参考，不构成投资建议！
理财有风险，投资需谨慎！！！
=====
```

(6) 添加股票

```
===== 添加股票 =====  
股票代码：000000  
股票名称：测试  
当前价格：1  
市盈率：00  
所属行业：测试  
请输入最近10天价格(按时间顺序，最早到最近)，用空格或回车分隔：  
12  
12  
12  
12  
12  
12  
12  
12  
12  
12  
股票数据已保存到 stocks.txt  
  
股票添加成功！  
  
按任意键继续...|
```

调试中存在的问题：

在设计数据展示界面的时候，文字标题和数据始终对不齐，导致错位。

解决办法:

使用 `setw` 函数对标题和数据设置位宽，并进行左对齐或者右对齐。

5.软件存在的问题与不足

1. 界面设计不是很人性化，没有使用真正的图形化界面
2. 安全性问题。可以直接看到文件中的用户数据
3. 录入数据麻烦，无法同时录入多个股票数据，智能逐个输入
4. 源代码中某些数字莫名其妙，不能给出合理的解释

解决思路

1. 未来可以使用 MFC 或者 Qt 来完成程序
2. 对存储文件进行加密
3. 增加一些工具函数，尝试导入表格数据
4. 对代码进行注释，提高代码质量

6. 设计体会与未来的展望

总结：以上就是我对我的系统的详细功能介绍，其中每一部分都有详细的功能介绍，希望老师再阅读量我的介绍之后科目明白我的思路。

其中有不少的算法和函数是我在原来的学习中么有接触过的，是查阅资料之后才写出来的，大多是根据学习资料依葫芦画瓢进行修改整理，希望不会成为这次课设的减分项。在日后，我也会继续学习相关技术，不断提升！！！！

7. 参考文献

B 站 vector 教程视频

[10 分钟让你拿捏动态数组 vector 哔哩哔哩 bilibili](#)

源代码仓库：https://github.com/Cosimo2233/curriculum_design.git